



Algoritmos voraces¹

Si los algoritmos voraces son la primera familia de algoritmos que examinaremos con detalle en este libro, la razón es sencilla: suelen ser los más fáciles. Tal como su nombre indica, el enfoque que aplican *es miope*, y toman decisiones basándose en la información que tienen disponible de modo inmediato, sin tener en cuenta los efectos que estas decisiones puedan tener en el futuro. Por tanto resultan *fáciles de inventar*, fáciles de implementar y, cuando funcionan, son eficientes. Sin embargo, como el mundo no suele ser tan sencillo, hay muchos problemas que no se pueden resolver correctamente con un enfoque tan grosero.

Los algoritmos voraces se utilizan típicamente para resolver problemas de optimización. Los ejemplos posteriores de este capítulo incluyen la búsqueda de la ruta más corta para ir desde un nodo a otro a través de una red de trabajo o la búsqueda del mejor orden para ejecutar un conjunto de tareas en una computadora. En tales contextos, un algoritmo voraz funciona seleccionando el arco, o la tarea, que parezca más prometedora en un determinado instante; nunca reconsidera su decisión, sea cual fuere la situación que pudiera surgir más adelante. No hay necesidad de evaluar alternativas, ni de emplear sofisticados procedimientos de seguimiento que permitan deshacer las decisiones anteriores. Comenzaremos este capítulo con un ejemplo cotidiano en el que esta táctica funciona bien.

6.1 DAR LA VUELTA (1)

Supongamos que vivimos en un país en el que están disponibles las siguientes monedas: 100 pesetas, 25 pesetas, 10 pesetas, 5 pesetas y 1 peseta. Nuestro problema consiste en diseñar un algoritmo *para pagar una cierta cantidad a un cliente*, utilizando el menor número posible de monedas. Por ejemplo, si tenemos que pagar 289 pesetas, la mejor solución consiste en dar al cliente 10 monedas: 2 de 100, 3 de 25, 1 de 10 y 4 de 1 peseta. Casi todos nosotros resolvemos este tipo de problema todos los días sin pensarlo dos veces, empleando de forma inconsciente un

algoritmo evidentemente voraz: empezamos por nada, y en cada fase vamos añadiendo a las monedas que ya estén seleccionadas una moneda de la mayor denominación posible, pero que no debe llevarnos más allá de la cantidad que haya que pagar.

El algoritmo se puede formalizar de la manera siguiente:

función *devolver cambio*(n): *conjunto de monedas*

{Da el cambio de n unidades utilizando el menor número posible de monedas. La constante C especifica las monedas disponibles}

const $C = \{100, 25, 10, 5, 1\}$

$S \leftarrow \emptyset$ { S es un conjunto que contendrá la solución}

$s \leftarrow 0$ { s es la suma de los elementos de S }

mientras $s \neq n$ **hacer**

$x \leftarrow$ el mayor elemento de C tal que $s + x \leq n$

si no existe ese elemento **entonces**

devolver «no encuentro la solución»

$S \leftarrow S \cup \{\text{una moneda de valor } x\}$

$s \leftarrow s + x$

devolver S

Es fácil convencerse (pero sorprendentemente difícil de probar formalmente) de que con los valores dados para las monedas, y disponiendo de un suministro adecuado de cada denominación, este algoritmo siempre produce una solución óptima para nuestro problema. Sin embargo, con una serie de valores diferente, o si el suministro de alguna de las monedas está limitado, el algoritmo voraz puede no funcionar; véanse los problemas 6.2 y 6.4. En algunos casos, puede seleccionar un conjunto de monedas que no sea óptimo (esto es, el conjunto contiene más monedas que las necesarias), mientras que quizá en otros casos no llegue a encontrar ninguna solución aun cuando ésta exista (aunque esto no puede suceder si disponemos de un suministro ilimitado de monedas de una unidad).

El algoritmo es «voraz» porque en cada paso selecciona la mayor de las monedas que pueda encontrar, sin preocuparse por lo correcto de esta decisión a la larga. Además, nunca cambia de opinión: una vez que una moneda se ha incluido en la solución, la tal moneda queda allí para siempre. Tal como explicaremos en la sección siguiente, éstas son las características de esta familia de algoritmos.

Para el problema particular de devolver cambio, se describe un algoritmo completamente distinto en el Capítulo 8. Este algoritmo alternativo emplea programación dinámica. El algoritmo de programación dinámica funciona siempre, mientras que el algoritmo voraz puede fallar; sin embargo, es menos comprensible que el algoritmo voraz y (cuando funcionan ambos algoritmos) resulta menos eficiente.

6.2 CARACTERÍSTICAS GENERALES DE LOS ALGORITMOS VORACES

Generalmente, los algoritmos voraces y los problemas que éstos resuelven *se caracterizan por la mayoría de las propiedades* siguientes (o quizá por todas):

Tenemos que resolver algún problema de forma óptima. Para construir la solución de nuestro problema, disponemos de un conjunto (o una lista) de candidatos: las monedas disponibles, las aristas de un grafo que se pueden utilizar para construir una ruta, el conjunto de tareas que hay que planificar, o lo que sea.

A medida que avanza el algoritmo, vamos acumulando dos conjuntos. Uno contiene candidatos que ya han sido considerados y seleccionados, mientras que el otro contiene candidatos que han sido considerados y rechazados.

Existe una función que comprueba si un cierto conjunto de candidatos constituye una *solución* de nuestro problema, ignorando si es o no óptima por el momento. Por ejemplo, ¿suman las monedas seleccionadas la cantidad que hay que pagar? ¿Proporcionan las aristas seleccionadas una ruta hasta el nodo que deseamos alcanzar? ¿Están planificadas todas las tareas?

Hay una segunda función que comprueba si un cierto conjunto de candidatos es *factible*, esto es, si es posible o no completar el conjunto añadiendo otros candidatos para obtener al menos una solución de nuestro problema. Una vez más, no nos preocupa aquí si esto es óptimo o no. Normalmente, se espera que el problema tenga al menos una solución que sea posible obtener empleando candidatos del conjunto que estaba disponible inicialmente.

Hay otra función más, la *función de selección*, que indica en cualquier momento cuál es el más prometedor de los candidatos restantes, que no han sido seleccionados ni rechazados.

Por último, existe una *función objetivo* que da el valor de la solución que hemos hallado: el número de monedas utilizadas para dar la vuelta, la longitud de la ruta que hemos construido, el tiempo necesario para procesar todas las tareas de la planificación, o cualquier otro valor que estemos intentando optimizar. A diferencia de las tres funciones mencionadas anteriormente, la función objetiva no aparece explícitamente en el algoritmo voraz.

Para resolver nuestro problema, busquemos un conjunto de candidatos que constituya una solución, y que optimice (minimice o maximice, según los casos) el valor de la *función objetivo*. Los algoritmos voraces avanzan paso a paso. Inicialmente, el conjunto de elementos seleccionados está vacío. Entonces, en cada paso se considera añadir a este conjunto el mejor candidato sin *considerar los restantes*, estando guiada nuestra elección por la función de selección. Si el conjunto ampliado de candidatos seleccionados ya no fuera factible, rechazamos el candidato que estamos considerando en ese momento. Sin embargo, si el conjunto aumentado sigue siendo factible, entonces añadimos el candidato actual al conjunto de candidatos selecciona-

dos, en donde pasará a estar desde ahora en adelante. Cada vez que se amplía el conjunto de candidatos seleccionados, comprobamos si éste constituye ahora una solución para nuestro problema. Cuando el algoritmo voraz funciona correctamente, la primera solución que se encuentre de esta manera es siempre óptima:

```
función voraz(C: conjunto): conjunto
    {C es el conjunto de candidatos}
    S ← ∅ {Construimos la solución en el conjunto S}
    mientras C ≠ ∅ y no solución(s) hacer
        x ← seleccionar(C)
        C ← C \ {x}
        si factible(S ∪ {x}) entonces S ← S ∪ {x}
    si solución(S) entonces devolver S
        sino devolver «no hay soluciones»
```

Está clara la razón por la cual tales algoritmos se denominan «voraces»: en cada paso, el procedimiento selecciona el mejor bocado que pueda tragar, sin preocuparse por el futuro. Nunca cambia de opinión: una vez que un candidato se ha incluido en la solución, queda allí para siempre; una vez que se excluye un candidato de la solución, nunca vuelve a ser considerado.

La función de selección suele estar relacionada con la función objetivo. Por ejemplo, si estamos intentando maximizar nuestros beneficios, es probable que seleccionemos aquel candidato restante que posea un mayor valor individual. Si intentamos minimizar el coste, entonces quizá seleccionemos el más barato de los candidatos disponibles, y así sucesivamente. Sin embargo, veremos que en algunas ocasiones puede haber varias funciones de selección plausibles, así que hay que seleccionar la adecuada si deseamos que nuestro algoritmo funcione correctamente.

Volviendo por un momento al ejemplo de dar cambio, lo que sigue es una forma de adecuar las características generales de los algoritmos voraces a las características particulares de este problema.

- ◊ Los candidatos son un conjunto de monedas, que representan en nuestro ejemplo 100, 25, 10, 5 y 1 pesetas, con tantas monedas de cada valor que nunca las agotamos. (Sin embargo, el conjunto de candidatos debe ser finito.)
- ◊ La función de solución comprueba si el valor de las monedas seleccionadas hasta el momento es *exactamente* el valor que hay que pagar.
- ◊ Un conjunto de monedas será *factible* si su valor total *no sobrepasa* la cantidad que haya que pagar.
- ◊ La función de selección toma la moneda de valor más alto que quede en el conjunto de candidatos.
- ◊ La función objetivo cuenta el número de monedas utilizadas en la solución.

Está claro que es más eficiente rechazar todas las monedas restantes de 100 pesetas (digamos) cuando el valor restante que hay que pagar cae por debajo de ese valor. El uso de la división entera para calcular cuántas monedas de un cierto valor hay que tomar también es más eficiente que actuar por sustracciones sucesivas. Si se adopta cualquiera de estas tácticas, entonces podemos relajar la condición consistente en que el conjunto de monedas debe ser finito.

6.3 GRAFOS: ÁRBOLES DE RECUBRIMIENTO MÍNIMO

Sea $G = \langle N, A \rangle$ un grafo conexo no dirigido en donde N es el conjunto de nodos y A es el conjunto de aristas. Cada arista posee una *longitud* no negativa. El problema consiste en hallar un subconjunto T de las aristas de G tal que utilizando solamente las aristas de T , todos los nodos deben quedar conectados, y además la suma de las longitudes de las aristas de T debe ser tan pequeña como sea posible. Dado que G es conexo, debe existir al menos una solución. Si G tiene aristas de longitud 0, pueden existir varias soluciones cuya longitud total sea la misma, pero que tengan números distintos de aristas. En este caso, dadas dos soluciones de igual longitud total, preferimos la que contenga menos aristas. Incluso con esta condición el problema puede tener varias soluciones diferentes y de igual valor. En lugar de hablar de las longitudes, podemos asociar un *coste* a cada arista. El problema, entonces, consiste en hallar un subconjunto T de las aristas cuyo coste total sea el menor posible. Evidentemente, este cambio de terminología no afecta a la forma en que resolvemos el problema.

Sea $G' = \langle N, T \rangle$ el grafo parcial formado por los nodos de G y las aristas de T , y supongamos que en N hay n nodos. Un grafo conexo con n nodos debe tener al menos $n - 1$ aristas, así que éste es el número mínimo de aristas que puede haber en T . Por otra parte, un grafo con n nodos y más de $n - 1$ aristas contiene al menos un ciclo; véase el Problema 6.7. Por tanto, si G' es conexo y T tiene más de $n - 1$ aristas, se puede eliminar al menos una arista sin desconectar G' , siempre y cuando seleccionemos una arista que forme parte de un ciclo. Esto hará o bien que disminuya la longitud total de las aristas de T , o bien que la longitud total quede intacta (si hemos eliminado una arista de longitud 0) a la vez que disminuye el número de aristas que hay en T . En ambos casos, la nueva solución es preferible a la anterior. Por tanto, un conjunto T con n o más aristas no puede ser óptimo. Se sigue que T debe tener exactamente $n - 1$ aristas, y como G' es conexo, tiene que ser un árbol.

El grafo G' se denomina *árbol de recubrimiento² mínimo* para el grafo G . Este problema tiene muchas aplicaciones. Por ejemplo, supongamos que los nodos de G representan ciudades, y sea el coste de una arista $\{a, b\}$ el coste de tender una línea telefónica desde a hasta b . Entonces un árbol de recubrimiento mínimo de G se corresponde con la red más barata posible para dar servicio a todas las ciudades en cuestión, siempre y cuando sólo se puedan utilizar conexiones directas entre ciudades (en otras palabras, siempre y cuando no se permita establecer centrales te-

lefónicas en el campo, *entre* las ciudades). Relajar esta condición es equivalente a permitir que se añadan nodos adicionales, auxiliares, a G . Esto puede permitirnos obtener soluciones más baratas: véase el problema 6.8.

A primera vista, parece que existen al menos dos líneas de ataque si deseamos encontrar un algoritmo voraz para este problema. Está claro que nuestro conjunto de candidatos debe ser el conjunto A de aristas que están en G . Una táctica posible consiste en comenzar por un conjunto vacío T , y seleccionar en cada etapa la arista más corta que todavía no haya sido seleccionada o rechazada, independientemente de donde se encuentra esta arista en G . Otra línea de ataque implica seleccionar un nodo y construir un árbol a partir de él, seleccionando en cada etapa la arista más corta posible que pueda extender el árbol hasta un nodo adicional. Es poco frecuente, pero ¡para este problema concreto, funcionan las dos aproximaciones! Antes de presentar los algoritmos, mostraremos la forma en que se aplica a este caso el esquema general de los algoritmos voraces, y presentaremos un lema que será de utilidad posteriormente:

- ◊ Los candidatos, como ya se ha indicado, son las aristas de G .
- ◊ Un conjunto de aristas es una solución si constituye un árbol de recubrimiento para los nodos de N .
- ◊ Un conjunto de aristas es factible si no contiene ningún ciclo.
- ◊ La función de selección que utilizamos varía con el algoritmo.
- ◊ La función objetivo que hay que minimizar es la longitud total de las aristas de la solución.

También necesitamos algo más de terminología. Diremos que un conjunto de aristas factible es *prometedor* si se puede extender para producir no sólo una solución, sino una solución óptima para nuestro problema. En particular, el conjunto vacío siempre es prometedor (puesto que siempre existe una solución óptima). Además, si un conjunto de aristas prometedor ya es una solución, la extensión requerida es irrelevante, y esta solución debe ser óptima. Decimos que una arista *sale* de un conjunto de nodos dado si esa arista tiene exactamente un extremo en el conjunto de nodos. Por tanto, una arista puede no salir de un conjunto dado de nodos bien porque ninguno de sus extremos esté en el conjunto, bien porque —y esto es menos evidente— sus dos extremos se encuentren en el conjunto. El lema siguiente es crucial para demostrar la corrección de los próximos algoritmos.

Lema 6.3.1. Sea $G = \langle N, A \rangle$ un grafo conexo no dirigido en el cual está dada la longitud de todas las aristas. Sea $B \subset N$ un subconjunto estricto de los nodos de G . Sea $T \subseteq A$ un conjunto prometedor de aristas tal que no haya ninguna arista de T que sale de B . Sea v la arista más corta que salga de B (o una de las más cortas si hay empates). Entonces $T \cup \{v\}$ es prometedor.

Demostración. Sea U un árbol de recubrimiento mínimo de G tal que $T \subseteq U$. Este U tiene que existir puesto que T es prometedor por hipótesis. Si $v \in U$, entonces no hay nada que probar. Si no, cuando añadimos v a U creamos exactamente un ciclo. (Ésta es una propiedad de los árboles: véase la Sección 5.5.) En este ciclo, puesto que v sale de B , existe necesariamente al menos otra arista u que también sale de B , o bien el ciclo no se cerraría; véase la figura 6.1. Si ahora eliminamos u , el ciclo desaparece y obtenemos un nuevo árbol V que abarca G . Sin embargo, la longitud de v , por definición, no es mayor que la longitud de u , y por tanto la longitud total de las aristas de V no sobrepasa la longitud total de las aristas de U . Por tanto, V es también un árbol de recubrimiento mínimo de G y contiene a v . Para completar la demostración, sólo hay que comentar que $T \subseteq V$ porque la arista u que se ha eliminado sale de B , y por tanto no podría haber sido una arista de T .

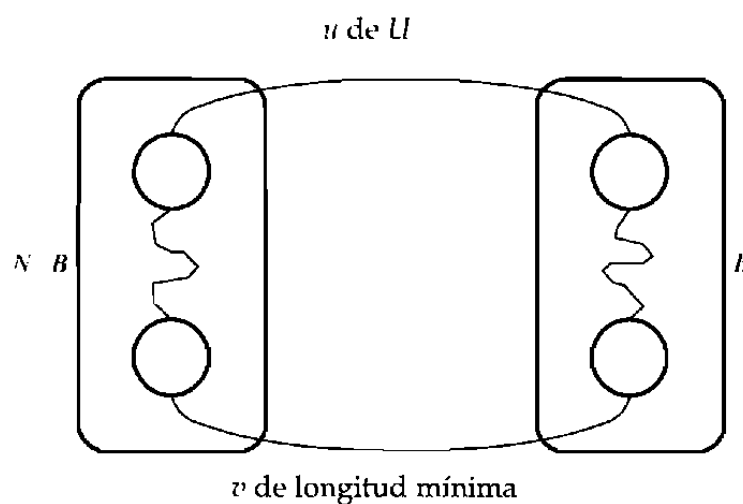


Figura 6.1. Se crea un ciclo si añadimos una arista v a U

6.3.1 Algoritmo de Kruskal

El conjunto de aristas T está vacío inicialmente. A medida que progresa el algoritmo, se van añadiendo aristas a T . Mientras no haya encontrado una solución, el grafo parcial formado por los nodos de G y las aristas de T consta de varios componentes conexos. (Inicialmente, cuando T está vacío, cada nodo de G forma una componente conexa distinta trivial.) Los elementos de T que se incluyen en una componente conexa dada forman un árbol de recubrimiento mínimo para los nodos de esta componente. Al final del algoritmo, sólo queda una componente conexa, así que T es un árbol de recubrimiento mínimo para todos los nodos de G .

Para construir componentes conexas más y más grandes, examinamos las aristas de G por orden creciente de longitudes. Si una arista une a dos nodos de compo-

nentes conexas distintas, se lo añadimos a T . Consiguientemente, las dos componentes conexas forman ahora una única componente. En caso contrario, se rechaza la arista: une a dos nodos de la misma componente conexa, y por tanto no se puede añadir a T sin formar un ciclo (porque las aristas de T forman un árbol para cada componente). El algoritmo se detiene cuando sólo queda una componente conexa.

Para ilustrar la forma en que funciona este algoritmo, considérese el gráfico de la figura 6.2. Por orden creciente de longitud, las aristas son: $\{1,2\}$, $\{2,3\}$, $\{4,5\}$, $\{6,7\}$, $\{1,4\}$, $\{2,5\}$, $\{4,7\}$, $\{3,5\}$, $\{2,4\}$, $\{3,6\}$, $\{5,7\}$ y $\{5,6\}$.

Paso	Arista considerada	Componentes conexas
Iniciación	—	$\{1\} \{2\} \{3\} \{4\} \{5\} \{6\} \{7\}$
1	$\{1, 2\}$	$\{1, 2\} \{3\} \{4\} \{5\} \{6\} \{7\}$
2	$\{2, 3\}$	$\{1, 2, 3\} \{4\} \{5\} \{6\} \{7\}$
3	$\{4, 5\}$	$\{1, 2, 3\} \{4, 5\} \{6\} \{7\}$
4	$\{6, 7\}$	$\{1, 2, 3\} \{4, 5\} \{6, 7\}$
5	$\{1, 4\}$	$\{1, 2, 3, 4, 5\} \{6, 7\}$
6	$\{2, 5\}$	rechazado
7	$\{4, 7\}$	$\{1, 2, 3, 4, 5, 6, 7\}$

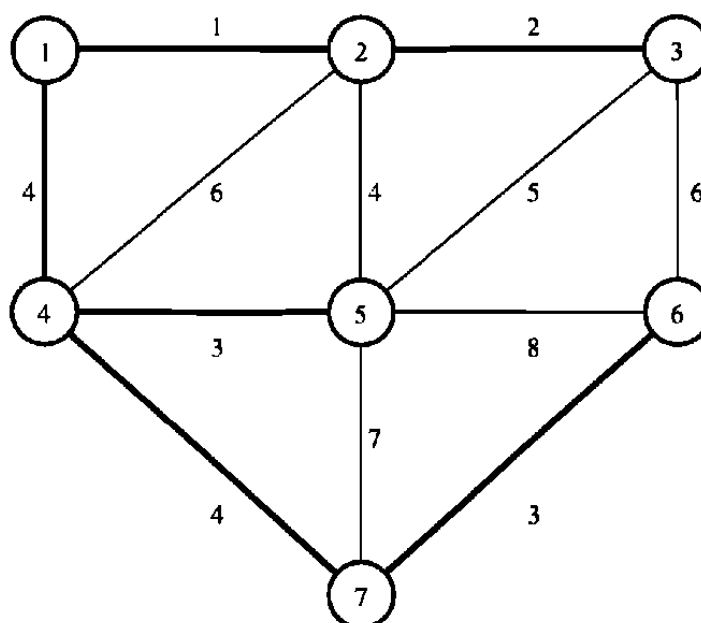


Figura 6.2 Un grafo y su árbol de recubrimiento mínimo

Cuando se detiene el algoritmo, T contiene las aristas seleccionadas $\{1,2\}$, $\{2,3\}$, $\{4,5\}$, $\{6,7\}$, $\{1,4\}$ y $\{4,7\}$. El árbol de recubrimiento mínimo se muestra mediante trazos gruesos en la figura 6.2; la longitud total es 17.

Teorema 6.3.2 *El algoritmo de Kruskal halla un árbol de recubrimiento mínimo*

Demostración. La demostración se hace por inducción matemática sobre el número de aristas que hay en el conjunto T . Mostraremos que si T es prometedor entonces sigue siendo prometedor en cualquier fase del algoritmo cuando se le añade una arista adicional. Cuando se detiene el algoritmo, T da una solución de nuestro problema; puesto que también es prometedora, la solución es óptima.

- ◊ *Base:* el conjunto vacío es prometedor porque G es conexo, y por tanto tiene que existir una solución.
- ◊ *Paso de inducción:* supongamos que T es prometedor inmediatamente antes de que el algoritmo añada una nueva arista $e = \{u, v\}$. Las aristas de T dividen a los nodos de G en dos o más componentes conexas; el nodo u se encuentra en una de estas componentes, y v está en otra componente distinta. Sea B el conjunto de nodos de esa componente que contiene a u . Ahora:
 - el conjunto B es un subconjunto estricto de los nodos de G (puesto que no incluye a v , por ejemplo);
 - T es un conjunto prometedor de aristas tal que ninguna arista de T sale de B (porque una arista de T tiene o bien ambas aristas en B , o bien ninguna arista en B , así que por definición no sale de B); y
 - e es una de las aristas más cortas que salen de B (porque todas las aristas estrictamente más cortas ya se han examinado, y o bien se han añadido a T , o bien se han rechazado porque tenían los dos extremos en la misma componente conexa).

Entonces se cumplen las condiciones del Lema 6.3.1, y concluimos que el conjunto $T \cup \{e\}$ también es prometedor.

Esto completa la demostración por inducción matemática de que el conjunto T es prometedor en todas las fases del algoritmo, y por tanto cuando se detiene el algoritmo, T no da meramente una solución de nuestro problema, sino una solución óptima.

Para implementar el algoritmo, tenemos que manejar un cierto número de conjuntos, a saber, los nodos de cada componente conexa. Es preciso efectuar rápidamente dos operaciones: *buscar*(x), que nos dice en qué componente conexa se encuentra el nodo x , y *fusionar*(A, B), para fusionar dos componentes conexas. Por tanto, utilizamos estructuras de partición; véase la Sección 5.9. Para este algoritmo, es preferible representar el grafo como un vector de aristas con sus longitudes asociadas, y no como una matriz de distancias; véase el problema 6.9. El algoritmo es el siguiente:

función *Kruskal*($G = \langle N, A \rangle$: grafo; longitud: $A \rightarrow \mathbb{R}^+$): conjunto de aristas
 {Iniciación}
 Ordenar A por longitudes crecientes
 $n \leftarrow$ el número de nodos que hay en N
 $T \leftarrow \emptyset$ {contendrá las aristas del árbol de recubrimiento mínimo}
 Iniciar n conjuntos, cada uno de los cuales contiene un elemento distinto de N
 {bucle voraz}
repetir
 $e \leftarrow \{u, v\} \leftarrow$ arista más corta, aun no considerada
 $comp_u \leftarrow buscar(u)$
 $comp_v \leftarrow buscar(v)$
 si $comp_u \neq comp_v$ **entonces**
 fusionar($comp_u, comp_v$)
 $T \leftarrow T \cup \{e\}$
hasta que T contenga $n - 1$ aristas
devolver T

Podemos evaluar el tiempo de ejecución del algoritmo en la forma siguiente. En un grafo con n nodos y a aristas, el número de operaciones está en:

- ◊ $\Theta(a \log a)$ para ordenar las aristas, lo cual es equivalente a $\Theta(a \log n)$ porque $n - 1 \leq a \leq n(n - 1)/2$
- ◊ $\Theta(n)$ para iniciar los n conjuntos disjuntos
- ◊ $\Theta(2a \alpha(2a, n))$ para todas las operaciones *buscar* y *fusionar*, en donde α es la función de crecimiento lento de la Sección 5.9 (esto se sigue de los resultados de la Sección 5.9, puesto que hay como máximo $2a$ operaciones *buscar* y $n - 1$ operaciones *fusionar* en un universo de n elementos), y
- ◊ $O(a)$ en el caso peor, para las operaciones restantes.

Concluimos que el tiempo total para el algoritmo está en $\Theta(a \log n)$ porque $\Theta(\alpha(2a, n)) \subset \Theta(\log n)$. Aunque esto no cambia el análisis en el caso peor, es preferible mantener las aristas en un montículo invertido (véase la Sección 7.5): de esta manera la arista más corta se encuentra en la raíz del montículo. Esto permite efectuar la inicialización en un tiempo que está en $\Theta(a)$, aun cuando cada búsqueda de un mínimo en el bucle **repetir** requiere ahora un tiempo que está en $\Theta(\log a) = \Theta(\log n)$. Esto resulta especialmente ventajoso si se encuentra el árbol de recubrimiento mínimo en un momento en el que quede por probar un número considerable de aristas. En tales casos, el algoritmo original desperdicia el tiempo ordenando estas aristas inútiles.

6.3.2 Algoritmo de Prim

En el algoritmo de Kruskal, la función de selección escoge las aristas por orden creciente de longitud, sin preocuparse demasiado por su conexión con las aristas seleccionadas anteriormente, salvo que se tiene cuidado para no formar nunca un ciclo. El resultado es un bosque de árboles que crece al azar, hasta que finalmente todas las componentes del bosque se fusionan en un único árbol. En el algoritmo de Prim, por

otra parte, el árbol de recubrimiento mínimo crece de forma natural, comenzando por una raíz arbitraria. En cada fase se añade una nueva rama al árbol ya construido; el algoritmo se detiene cuando se han alcanzado todos los nodos.

Sea B un conjunto de nodos, y sea T un conjunto de aristas. Inicialmente, B contiene un único nodo arbitrario, y T está vacío. En cada paso, el algoritmo de Prim busca la arista más corta posible $\{u, v\}$ tal que $u \in B$ y $v \in N \setminus B$. Entonces añade v a B y $\{u, v\}$ a T . De esta manera las aristas de T forman en todo momento un árbol de recubrimiento mínimo para los nodos de B . Continuamos mientras $B \neq N$. Lo que sigue es un enunciado informal del algoritmo:

función $\text{Prim}(G = \langle N, A \rangle; \text{grafo}; \text{longitud } A \rightarrow \mathbb{R}^+)$: conjunto de aristas.

{Iniciación}

$T \leftarrow \emptyset$

$B \leftarrow \{\text{un miembro arbitrario de } N\}$

mientras $B \neq N$ **hacer**

 buscar $e = \{u, v\}$ de longitud mínima tal que
 $u \in B$ y $v \in N \setminus B$

$T \leftarrow T \cup \{e\}$

$B \leftarrow B \cup \{v\}$

devolver T

Para ilustrar este algoritmo, considérese una vez más el grafo de la figura 6.2. Arbitrariamente, seleccionamos el nodo uno como nodo inicial. Ahora el algoritmo podría progresar en la forma siguiente:

Paso	$\{u, v\}$	B
Inicialización	—	$\{1\}$
1	$\{1, 2\}$	$\{1, 2\}$
2	$\{2, 3\}$	$\{1, 2, 3\}$
3	$\{1, 4\}$	$\{1, 2, 3, 4\}$
4	$\{4, 5\}$	$\{1, 2, 3, 4, 5\}$
5	$\{4, 7\}$	$\{1, 2, 3, 4, 5, 7\}$
6	$\{7, 6\}$	$\{1, 2, 3, 4, 5, 6, 7\}$

Cuando se detiene el algoritmo, T contiene las aristas seleccionadas $\{1, 2\}$, $\{2, 3\}$, $\{1, 4\}$, $\{4, 5\}$, $\{4, 7\}$ y $\{7, 6\}$. La demostración de que el algoritmo funciona es parecida a la demostración del algoritmo de Kruskal.

Teorema 6.3.3 *El algoritmo de Prim halla un árbol de recubrimiento mínimo*

Demostración. La demostración es por inducción matemática sobre el número de aristas que hay en el conjunto T . Demostraremos que si T es prometedor en alguna fase del algoritmo,

entonces sigue siendo prometedor al añadir una arista adicional. Cuando se detiene el algoritmo, T da una solución para nuestro problema; puesto que también es prometedora, la solución es óptima.

◊ *Base*: el conjunto vacío es prometedor.

◊ *Paso de inducción*: suponga que T es prometedor inmediatamente antes de que el algoritmo añada una nueva arista $e = \{u, v\}$. Ahora B es un subconjunto estricto de N (porque el algoritmo se detiene cuando $B = N$), T es un conjunto de aristas prometedor por hipótesis de inducción, y e es por definición una de las aristas más cortas que salen de B . Entonces las condiciones del lema 6.3.1 se cumplen, y $T \cup \{e\}$ también es prometedor.

Esto completa la demostración por inducción matemática de que el conjunto T es prometedor en todas las fases del algoritmo. Por tanto, cuando se detiene el algoritmo, T ofrece una solución óptima de nuestro problema.

Para obtener una implementación sencilla en una computadora, supongamos que los nodos de G están numerados de 1 a n , así que $N = \{1, 2, \dots, n\}$. Supongamos también que hay una matriz simétrica L que da la longitud de todas las aristas, con $L[i, j] = \infty$ si no existe la arista correspondiente. Utilizaremos dos matrices. Para todo nodo $i \in N \setminus B$, *más próximo*[i] proporciona el nodo de B que está más próximo a i , y *distmin*[i] da la distancia desde i hasta este nodo más próximo. Para un nodo $i \in B$, hacemos *distmin*[i] = -1. (De esta manera, podemos saber si un nodo está o no en B .) El conjunto B , que recibe el valor inicial arbitrario $\{1\}$, no se representa explícitamente; *más próximo*[1] y *distmin*[1] nunca se utilizan. Véase el algoritmo:

función *Prim*($L[1..n, 1..n]$): conjunto de aristas

{inicialización: sólo el nodo 1 se encuentra en B }

$T \leftarrow \emptyset$ {contendrá las aristas del árbol de recubrimiento mínimo}

para $i \leftarrow 2$ **hasta** n **hacer**

más próximo[i] $\leftarrow 1$

distmin[i] $\leftarrow L[i, 1]$

{bucle voraz}

repetir $n - 1$ **veces**

mín $\leftarrow \infty$

para $j \leftarrow 2$ **hasta** n **hacer**

si $0 \leq \text{distmin}[j] < \text{mín}$ **entonces** *mín* $\leftarrow \text{distmin}[j]$

$k \leftarrow j$

$T \leftarrow T \cup \{\{\text{más próximo}[k], k\}\}$

distmin[k] $\leftarrow -1$ {se añade k a B }

para $j \leftarrow 2$ **hasta** n **hacer**

si $L[j, k] < \text{distmin}[j]$ **entonces** *distmin*[k] $\leftarrow L[j, k]$

más próximo[j] $\leftarrow k$

devolver T

El bucle principal del algoritmo se ejecuta $n - 1$ veces; en cada iteración, el bucle **para** anidado requiere un tiempo que está en $\Theta(n)$. Por tanto, el algoritmo de Prim requiere un tiempo que está en $\Theta(n^2)$.

Se vio anteriormente que el algoritmo de Kruskal requiere un tiempo que está en $\Theta(a \log n)$, en donde a es el número de aristas que hay en el grafo. Para un grafo denso, a tiende a $n(n-1)/2$. En este caso, el algoritmo de Kruskal requiere un tiempo que se encuentra en $\Theta(n^2 \log n)$, y el algoritmo de Prim puede ser mejor. Para un grafo disperso, a tiende a n . En este caso, el algoritmo de Kruskal requiere un tiempo que está en $\Theta(n \log n)$, y el algoritmo de Prim, tal como se ha presentado aquí, es probablemente menos eficiente. Sin embargo, el algoritmo de Prim, al igual que el de Kruskal, se puede implementar utilizando montículos. En este caso —una vez más, como el algoritmo de Kruskal— requiere un tiempo que está en $\Theta(a \log n)$. Existen otros algoritmos más eficientes que el de Prim o el de Kruskal; véase la Sección 6.8.

6.4 GRAFOS: CAMINOS MÍNIMOS

Considere ahora un grafo dirigido $G = \langle N, A \rangle$ en donde N es el conjunto de nodos de G , y A es el conjunto de aristas dirigidas. Cada arista posee una *longitud* no negativa. Se toma uno de los nodos como nodo *origen*. El problema consiste en determinar la longitud del camino mínimo que va desde el origen hasta cada uno de todos los demás nodos del grafo. Tal como sucedía en la Sección 6.3, podríamos hablar igualmente bien acerca del *coste* de una arista, en lugar de mencionar su longitud, y se podría plantear el problema consistente en determinar la ruta más barata desde el origen hasta cada uno de todos los demás nodos.

Este problema se puede resolver mediante un algoritmo voraz que recibe frecuentemente el nombre de *algoritmo de Dijkstra*. El algoritmo utiliza dos conjuntos de nodos, S y C . En todo momento, el conjunto S contiene aquellos nodos que ya han sido seleccionados; como veremos, la distancia mínima desde el origen ya es conocida para todos los nodos de S . El conjunto C contiene todos los demás nodos, cuya distancia mínima desde el origen todavía no es conocida, y que son candidatos a ser seleccionados en alguna etapa posterior. Por tanto, tenemos la propiedad invariante $N = S \cup C$. En un primer momento, S contiene nada más el origen en sí; cuando se detiene el algoritmo, S contiene todos los nodos del grafo y nuestro problema está resuelto. En cada paso seleccionamos aquel nodo de C cuya distancia al origen sea mínima, y se lo añadimos a S .

Diremos que un camino desde el origen hasta algún otro nodo es *especial* si todos los nodos intermedios a lo largo del camino pertenecen a S . En cada fase del algoritmo, hay una matriz D que contiene la longitud del camino especial más corta que va hasta cada nodo del grafo. En el momento en que se desea añadir un nuevo nodo v a S , el camino especial más corto hasta v es también el más corto de los caminos posibles hasta v (esto se demostrará más adelante). Cuando se detiene el algoritmo, todos los nodos del grafo se encuentran en S , y por tanto todos los caminos desde el origen hasta algún otro nodo son especiales. Consiguientemente, los valores que hay en D dan la solución del problema de caminos mínimos.

```

función Dijkstra( $L[1..n, 1..n]$ ): matriz  $[2..n]$ 
    matriz  $D[2..n]$ 
    {Iniciación}
     $C \leftarrow \{2, 3, \dots, n\}$  { $S = N \setminus C$  sólo existe implícitamente}
    para  $i \leftarrow 2$  hasta  $n$  hacer  $D[i] \leftarrow L[1, i]$ 
    {bucle voraz}
    repetir  $n - 2$  veces
         $v \leftarrow$  algún elemento de  $C$  que minimiza  $D[v]$ 
         $C \leftarrow C \setminus \{v\}$  {e implícitamente  $S \leftarrow S \cup \{v\}$ }
        para cada  $w \in C$  hacer
             $D[w] \leftarrow \min(D[w], D[v] + L[v, w])$ 
    devolver  $D$ 

```

Paso	r	C	D
Inicialización	—	{2, 3, 4, 5}	[50, 30, 100, 10]
1	5	{2, 3, 4}	[50, 30, 20, 10]
2	4	{2, 3}	[40, 30, 20, 10]
3	3	{2}	[35, 30, 20, 10]

Para determinar no solamente la longitud de los caminos mínimos sino también por dónde pasan, se añade una segunda matriz $P[2..n]$, en donde $P[v]$ contiene el número del nodo que precede a v dentro del camino más corto. Para hallar el camino más corto, se siguen los punteros P hacia atrás, desde el destino hacia el origen. Las modificaciones necesarias para el algoritmo son sencillas:

si $D[w] > D[v] + L[v, w]$ **entonces** $D[w] \leftarrow D[v] + L[v, w]$
 $P[w] \leftarrow v$

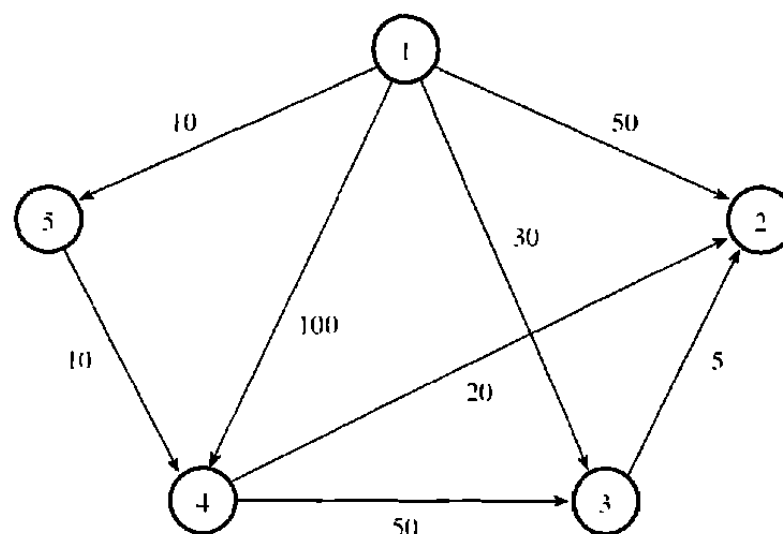


Figura 6.3. Un grafo dirigido

La demostración de que el algoritmo funciona es, una vez más, por inducción matemática.

Teorema 6.4.1 *El algoritmo de Dijkstra halla los caminos más cortos desde un único origen hasta los demás nodos del grafo*

Demostración. Demostraremos por inducción matemática que:

- (a) Si un nodo $i \neq 1$ está en S , entonces $D[i]$ da la longitud del camino más corto desde el origen hasta i , y
- (b) si un nodo i no está en S , entonces $D[i]$ da la longitud del camino *especial* más corto desde el origen hasta i .

◊ *Base:* inicialmente, sólo el nodo 1, que es el origen, se encuentra en S , así que la situación (a) es cierta sin más demostración. Para los demás nodos, el único camino especial desde el origen es el camino directo, y D recibe valores iniciales en consecuencia. Por tanto la situación (b) también es cierta cuando comienza el algoritmo.

◊ *Hipótesis de inducción:* la hipótesis de inducción es que tanto la situación (a) como la situación (b) son válidas inmediatamente antes de añadir un nodo v a S . Detallamos por separado los pasos de inducción para las situaciones (a) y (b).

◊ *Paso de inducción para la situación (a):* para todo nodo que ya esté en S antes de añadir v , no cambia nada, así que la situación (a) sigue siendo válida. En cuanto al nodo v , ahora pertenecerá a S . Antes de añadirlo a S , es preciso comprobar que $D[v]$ proporcione la longitud del camino más corto que va desde el origen hasta v . Por hipótesis de inducción, nos da ciertamente la longitud del camino *especial* más corto. Por tanto, hay que verificar que el camino más corto desde el origen hasta v no pase por ninguno de los nodos que no pertenecen a S .

Supongamos lo contrario; esto es, supongamos que cuando se sigue el camino más corto desde el origen hasta v , se encuentran uno o más nodos (sin contar el propio v)

que no pertenecen a S . Sea x el primer nodo encontrado con estas características; véase la figura 6.4. Ahora el segmento inicial de esta ruta, hasta llegar a x , es una ruta especial, así que la distancia hasta x es $D[x]$ por la parte (b) de la hipótesis de inducción. Claramente la distancia total hasta v a través de x no es más corta que este valor, porque las longitudes de las aristas son no negativas. Finalmente, $D[x]$ no es menor que $D[v]$, porque el algoritmo ha seleccionado a v antes que a x . Por tanto, la distancia total hasta v a través de x es como mínimo $D[v]$, y el camino a través de x no puede ser más corto que el camino especial que lleva hasta v .

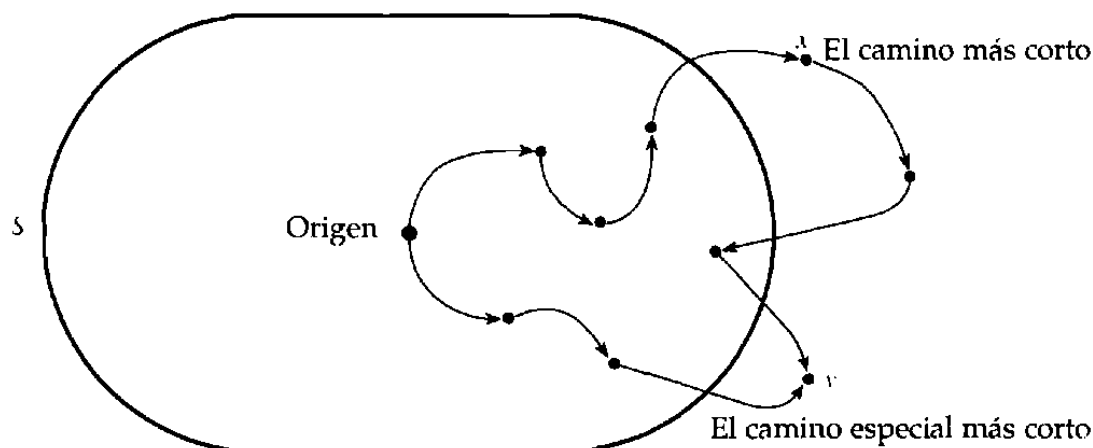


Figura 6.4. El camino más corto hasta v no puede visitar x

Por tanto, hemos verificado que cuando se añade v a S , la parte (a) de la inducción sigue siendo válida.

◊ *Paso de inducción para la situación (b):* considérese ahora un nodo w , distinto de v , que no se encuentre en S . Cuando v se añade a S , hay dos posibilidades para el camino especial más corto desde el origen hasta w : o bien no cambia, o bien ahora pasa a través de v (y posiblemente, también a través de otros nodos de S). En el segundo caso, sea x el último nodo de S visitado antes de llegar a w . La longitud de este camino es $D[x] + L[x, w]$. Parece a primera vista que para calcular el nuevo valor de $D[w]$ debiéramos comparar el valor anterior de $D[w]$ con los valores de $D[x] + L[x, w]$ para todo nodo x de S (incluyendo a v). Sin embargo, para todos los nodos x de S salvo v , esta comparación se ha hecho cuando se añadió x a S , y $D[x]$ no ha variado desde entonces. Por tanto, el nuevo valor de $D[w]$ se puede calcular sencillamente comparando el valor anterior con $D[v] + L[v, w]$.

Puesto que el algoritmo hace esto explícitamente, asegura que la parte (b) de la inducción siga siendo cierta también cuando se añade a S un nuevo nodo v .

Para completar la demostración de que el algoritmo funciona, obsérvese que cuando se detenga el algoritmo, todos los nodos menos uno estarán en S (aun cuando el conjunto S no se construye explícitamente). En ese momento queda claro que el camino más corto desde el origen hasta el nodo restante es un camino especial.

Análisis del algoritmo. Supongamos que el algoritmo de Dijkstra se aplica a un grafo que posee n nodos y a aristas. Utilizando la representación sugerida hasta el momento, este caso se da en la forma de una matriz $L[1..n, 1..n]$. La iniciación requiere un tiempo que está en $O(n)$. En una implementación directa, la selección de v dentro del bucle (**repetir**) requiere examinar todos los elementos de C , así que examinaremos $n - 1, n - 2, \dots, 2$ valores de D en las sucesivas iteraciones, dando un tiempo total que está en $\Theta(n^2)$. El bucle **para (desde)** interno realiza $n - 2, n - 3, \dots, 1$ iteraciones dando también un tiempo total que está en $\Theta(n^2)$. El tiempo requerido por esta versión del algoritmo está, por tanto, en $\Theta(n^2)$.

Si $a \ll n^2$, podríamos tener la esperanza de evitar examinar las muchas entradas que contienen ∞ en la matriz L . Teniendo esto en cuenta, sería preferible representar el grafo mediante una matriz de n listas, que diera para cada nodo su distancia directa a los nodos adyacentes (como el tipo *listagraf* de la Sección 5.4). Esto nos permite ahorrar tiempo en el bucle **para** interno, porque sólo hay que considerar aquellos nodos w que sean adyacentes a v ; ahora bien, ¿cómo podemos evitar que se necesite un tiempo en $\Omega(n^2)$ para determinar sucesivamente los $n - 2$ valores que va tomando v ?

La respuesta consiste en utilizar un montículo invertido que contiene un nodo para cada elemento v de C , ordenado por el valor de $D[v]$. De esta manera, el elemento v de C que minimiza $D[v]$ siempre se encontrará en la raíz. La iniciación del montículo requiere un tiempo que está en $\Theta(n)$. La instrucción « $C \leftarrow C \setminus \{v\}$ » consiste en eliminar la raíz del montículo, lo cual requiere un tiempo que se encuentra en $O(\log n)$. En cuanto al bucle **para** interno, ahora consiste en estudiar, para cada elemento w de C adyacente a v , si $D[v] + L[v, w]$ es menor que $D[w]$. En tal caso, debemos modificar $D[w]$ y filtrar a w dentro del montículo, lo cual requiere una vez más un tiempo que está en $O(\log n)$. Esto no sucede más que una vez para cada arista del grafo.

Para resumir, tenemos que eliminar la raíz del montículo exactamente $n - 2$ veces, y tenemos que flotar un máximo de a nodos, lo cual da un tiempo total que está en $\Theta((a + n) \log n)$. Si el grafo es conexo, $a \geq n - 1$, y el tiempo se encuentra en $\Theta(a \log n)$. La implementación sencilla es preferible, por tanto, si el grafo es denso, mientras que resulta preferible utilizar un montículo si el grafo es disperso. Si $a \in \Theta(n^2 / \log n)$, entonces la elección de la representación puede depender de la implementación concreta utilizada. El Problema 6.16 sugiere una forma de acelerar el algoritmo, empleando un montículo k -ario con un valor de k bien seleccionado; se conocen otros algoritmos aún más rápidos; véanse el Problema 6.17 y la Sección 6.8.

6.5 EL PROBLEMA DE LA MOCHILA (1)

Este problema surge de distintas maneras. En este capítulo examinaremos la versión más sencilla; en la Sección 8.4 se presentará una variante más difícil.

Nos dan n objetos y una mochila. Para $i = 1, 2, \dots, n$, el objeto i tiene un peso positivo w_i y un valor positivo v_i . La mochila puede llevar un peso que no sobrepasa

se W . Nuestro objetivo es llenar la mochila de tal manera que se maximice el valor de los objetos transportados, respetando la limitación de capacidad impuesta. En esta primera versión del problema, suponemos que se pueden romper los objetos en trozos más pequeños, de manera que podamos decidir llevar solamente una fracción x_i del objeto i , con $0 \leq x_i \leq 1$. (Si no se nos permite romper los objetos, el problema es mucho más difícil.) En este caso, el objeto i contribuye en $x_i w_i$ al peso total de la mochila, y en $x_i v_i$ al valor de la carga. En símbolos, el problema se puede enunciar en la forma siguiente:

$$\text{maximizar } \sum_{i=1}^n x_i v_i \text{ con la restricción } \sum_{i=1}^n x_i w_i \leq W$$

donde $v_i > 0$, $w_i > 0$ y $0 \leq x_i \leq 1$ para $1 \leq i \leq n$. Aquí las condiciones que afectan a v_i y a w_i son restricciones sobre el problema; las de x_i son restricciones sobre la solución. Utilizaremos un algoritmo voraz para resolver el problema. En términos de nuestro esquema general, los candidatos son los diferentes objetos, y la solución es un vector $(x_1, \dots, x_n)$ que nos dice qué fracción de cada objeto hay que incluir. Una solución será factible cuando se respetan las restricciones indicadas anteriormente, y la función objetivo es el valor total de los objetos que están en la mochila. Queda por ver qué debemos tomar como función de selección.

Si $\sum_{i=1}^n w_i \leq W$, está claro que lo óptimo es meter todos los objetos dentro de la mochila. Por tanto, podemos asumir que en los casos interesantes del problema, es $\sum_{i=1}^n w_i > W$. También está claro que una solución óptima debe llenar exactamente la mochila, porque en caso contrario se podría añadir una fracción de alguno de los objetos restantes e incrementar el valor de la carga. Por tanto, en una solución óptima $\sum_{i=1}^n x_i w_i = W$. Puesto que esperamos encontrar un algoritmo voraz que funcione, nuestra estrategia general consistirá en seleccionar cada objeto por turno en algún orden adecuado, poner la mayor fracción posible del objeto seleccionado en la mochila, y detenernos cuando la mochila esté llena. Véase el algoritmo:

```

función mochila( $w[1..n]$ ,  $v[1..n]$ ,  $W$ ): matriz  $[1..n]$ 
  {Inicialización}
  para  $i = 1$  hasta  $n$  hacer  $x[i] \leftarrow 0$ 
   $\text{peso} \leftarrow 0$ 
  {bucle voraz}
  mientras  $\text{peso} < W$  hacer
     $i \leftarrow$  el mejor objeto restante {ver más abajo}
    si  $\text{peso} + w[i] \leq W$  entonces    $x[i] \leftarrow 1$ 
                                      $\text{peso} \leftarrow \text{peso} + w[i]$ 
    sino  $x[i] \leftarrow (W - \text{peso}) / w[i]$ 
         $\text{peso} \leftarrow W$ 

  devolver  $x$ 
```

Para este problema hay por lo menos tres funciones de selección posibles: en cada fase, podemos seleccionar el objeto más valioso, argumentando que esto incrementa el valor de la carga del modo más rápido posible; podemos seleccionar el objeto más pequeño restante, basándonos en que de este modo la capacidad se agota de la forma más lenta posible; o bien podemos evitar estos extremos seleccionando aquel objeto cuyo valor por unidad de peso sea el mayor posible. Las figuras 6.5 y 6.6 muestran la forma en que funcionan estas tácticas en un caso particular. Aquí tenemos cinco objetos y $W = 100$. Si seleccionamos los objetos por orden decreciente de valores, entonces seleccionaremos primero el objeto 3, luego el 5 y finalmente llenaremos la mochila con la mitad del objeto 4. El valor de la solución obtenida de esta manera es de $66 + 60 + 40/2 = 146$. Si seleccionamos los objetos por orden de peso creciente, entonces seleccionaremos los objetos 1, 2, 3 y 4 por este orden, y la mochila estará llena. El valor de la solución es de $20 + 30 + 66 + 40 = 156$. Finalmente, si seleccionamos los objetos por orden decreciente de v_i/w_i , entonces seleccionaremos primero el objeto 3, luego el objeto 1, después el objeto 2, y finalmente llenaremos la mochila con cuatro quintos del objeto 5. Empleando esta táctica, el valor de la solución es $20 + 30 + 66 + 0,8 \times 60 = 164$.

	$n = 5, W = 100$				
w	10	20	30	40	50
v	20	30	66	40	60
v/w	2.0	1.5	2.2	1.0	1.2

Figura 6.5. Un ejemplar para el problema de la mochila

Este ejemplo muestra que la solución obtenida mediante un algoritmo voraz que maximiza el valor de los objetos seleccionados no es necesariamente óptimo, ni tampoco lo es la solución obtenida minimizando el peso de los objetos seleccionados.

Seleccionar:	i	x_i	Valor
Máx v_i	0	0	146
Mín w_i	1	1	156
Máx v_i/w_i	1	1	164

Figura 6.6. Tres enfoques voraces para el ejemplar de la figura 6.5

Afortunadamente la demostración siguiente muestra que la tercera posibilidad, que consiste en seleccionar el objeto que maximice el valor por unidad de peso, lleva realmente a una solución óptima.

Teorema 6.5.1 Si se seleccionan los objetos por orden decreciente de v_i/w_i , entonces el algoritmo de la mochila encuentra una solución óptima

Demostración. Supongamos, sin pérdida de generalidad, que los objetos disponibles están ordenados por valor decreciente de coste por unidad de peso, esto es, que

$$v_1 / w_1 \geq v_2 / w_2 \geq \dots \geq v_n / w_n$$

Sea $X = (x_1, x_2, \dots, x_n)$ la solución hallada por el algoritmo voraz. Si todos los x_i son iguales a 1, entonces esta solución es claramente óptima. En caso contrario, supongamos que j denota el menor índice tal que $x_j < 1$. Examinando la forma en que funciona el algoritmo, está claro que $x_i = 1$ cuando $i < j$, que $x_i = 0$ cuando $i > j$, y que $\sum_{i=1}^n x_i w_i = W$. Sea $V(X) = \sum_{i=1}^n x_i v_i$ el valor de la solución X .

Ahora sea $Y = (y_1, \dots, y_n)$ cualquier solución factible. Como Y es factible, $\sum_{i=1}^n y_i w_i \leq W$ y por tanto $\sum_{i=1}^n (x_i - y_i) w_i \geq 0$. Sea $V(Y) = \sum_{i=1}^n y_i v_i$ el valor de la solución Y . Ahora

$$V(X) - V(Y) = \sum_{i=1}^n (x_i - y_i) v_i = \sum_{i=1}^n (x_i - y_i) w_i \frac{v_i}{w_i}$$

Cuando $i < j$, $x_i = 1$ y por tanto $x_i - y_i$ es positivo o nulo, mientras que $v_i / w_i \geq v_j / w_j$; cuando $i > j$, $x_i = 0$ y por tanto $x_i - y_i$ es negativo o nulo, mientras que $v_i / w_i \leq v_j / w_j$; y por supuesto cuando $i = j$, $v_i / w_i = v_j / w_j$. Por tanto, en todos los casos se tiene que $(x_i - y_i)(v_i / w_i) \geq (x_i - y_i)(v_j / w_j)$. Por tanto:

$$V(X) - V(Y) \geq (v_j / w_j) \sum_{i=1}^n (x_i - y_i) w_i \geq 0$$

Con lo cual, hemos demostrado que ninguna solución factible puede tener un valor mayor que $V(X)$, por lo que la solución X es óptima.

La implementación del algoritmo es directa. Si los objetos ya están ordenados por orden decreciente de v_i / w_i , entonces está claro que el algoritmo voraz requiere un tiempo que está en $O(n)$; el tiempo total incluyendo la ordenación se encuentra por tanto en $O(n \log n)$. Tal como sucedía en la Sección 6.3.1, puede merecer la pena almacenar los objetos en un montículo, con el mayor valor de v_i / w_i en la raíz. La creación del montículo requiere un tiempo que está en $O(n)$, mientras que cada pasada por el bucle voraz requiere un tiempo que está en $O(\log n)$, puesto que la propiedad del montículo debe ser restaurada después de eliminar la raíz. Aun cuando esto no altera el análisis en el caso peor, puede resultar más rápido si sólo se necesitan unos pocos objetos para llenar la mochila.

6.6 PLANIFICACIÓN

En esta sección presentamos dos problemas que conciernen a la forma óptima de planificar tareas en una sola máquina. En el primero, el problema consiste en mi-

minimizar el tiempo medio que invierte cada tarea en el sistema. En el segundo, las tareas tienen un plazo fijo de ejecución, y cada tarea aporta unos ciertos beneficios sólo si está acabada al llegar su plazo: nuestro objetivo es maximizar la rentabilidad. Ambos problemas se pueden resolver empleando algoritmos voraces.

6.6.1 Minimización del tiempo en el sistema

Un único servidor, como por ejemplo un procesador, un surtidor de gasolina, o un cajero de un banco, tiene que dar servicio a n clientes. El tiempo requerido por cada cliente se conoce de antemano: el cliente i requerirá un tiempo t_i para $1 \leq i \leq n$. Deseamos minimizar el tiempo medio invertido por cada cliente en el sistema. Dado que el número n de clientes está predeterminado, esto equivale a minimizar el tiempo total invertido en el sistema por todos los clientes. En otras palabras, deseamos minimizar

$$T = \sum_{i=1}^n (\text{tiempo en el sistema para el cliente } i)$$

Supongamos por ejemplo que tenemos tres clientes, con $t_1 = 5$, $t_2 = 10$ y $t_3 = 3$. Existen seis órdenes de servicio posibles:

Orden	T
1 2 3:	$5 + (5 + 10) + (5 + 10 + 3) = 38$
1 3 2:	$5 + (5 + 3) + (5 + 3 + 10) = 31$
2 1 3:	$10 + (10 + 5) + (10 + 5 + 3) = 43$
2 3 1:	$10 + (10 + 3) + (10 + 3 + 5) = 41$
3 1 2:	$3 + (3 + 5) + (3 + 5 + 10) = 29 \quad \leftarrow \text{óptimo}$
3 2 1:	$3 + (3 + 10) + (3 + 10 + 5) = 34$

En el primer caso, se sirve inmediatamente al cliente 1, el cliente 2 espera mientras se sirve al cliente 1 y entonces le llega el turno, y el cliente 3 espera mientras se sirve a los clientes 1 y 2, y se le sirve en último lugar; el tiempo total invertido en el sistema por los tres clientes es 38. Los cálculos para los demás casos son similares.

En este caso, la planificación óptima se obtiene cuando se sirve a los tres clientes por orden creciente de tiempos de servicio: el cliente 3, que necesita el menor tiempo, es servido en primer lugar, mientras que el cliente 2, que necesita mayor tiempo, es servido en último lugar. Al servir a los clientes por orden decreciente de tiempos de servicio se obtiene la peor planificación. Sin embargo, un ejemplo no es una demostración de que siempre sea así.

Para dar plausibilidad a la idea de que puede ser óptimo planificar los clientes por orden creciente de tiempo de servicio, imaginemos un algoritmo voraz que construye la planificación óptima elemento a elemento. Supongamos que después de planificar el servicio para los clientes $i_1, i_2, \dots, i_m$ se añade un cliente j . El incre-

mento del tiempo T en esta fase es igual a la suma de los tiempos de servicio para los clientes desde i_1 hasta i_m (porque esto es lo que tiene que esperar el cliente j antes de recibir servicio) más t_j , que es el tiempo necesario para servir al cliente j . Para minimizar esto, dado que un algoritmo voraz nunca reconsidera sus decisiones anteriores, lo único que podemos hacer es minimizar t_j . Nuestro algoritmo voraz, por tanto, es bastante sencillo: en cada paso se añade al final de la planificación al cliente que requiera el menor servicio de entre los restantes.

Teorema 6.6.1 *El algoritmo voraz es óptimo*

Demostración. Sea $P = p_1 p_2 \dots p_n$ cualquier permutación de enteros del 1 al n , y sea $s_i = t_{p_i}$. Si se sirven clientes en el orden P , entonces el tiempo de servicio requerido por el i -ésimo cliente que haya que servir será s_i , y el tiempo total transcurrido en el sistema por todos los clientes es

$$\begin{aligned} T(P) &= s_1 + (s_1 + s_2) + (s_1 + s_2 + s_3) + \dots \\ &= ns_1 + (n-1)s_2 + (n-2)s_3 + \dots \\ &= \sum_{k=1}^n (n-k+1)s_k \end{aligned}$$

Supongamos ahora que P no organiza a los clientes por orden de tiempos crecientes de servicio. Entonces se pueden encontrar dos enteros a y b con $a < b$ y $s_a > s_b$. En otras palabras, se sirve al a -ésimo cliente antes que al b -ésimo, aun cuando el primero necesite más tiempo de servicio que el segundo; véase la figura 6.7. Si intercambiamos la posición de estos dos clientes, obtendremos un nuevo orden de servicio P' que es simplemente el orden P después de intercambiar los enteros p_a y p_b . El tiempo total transcurrido pasado en el sistema por todos los clientes si se emplea la planificación P' es

$$T(P') = (n-a+1)s_b + (n-b+1)s_a + \sum_{\substack{k=1 \\ k \neq a, b}}^n (n-k+1)s_k$$

La planificación nueva es preferible a la vieja, porque

$$\begin{aligned} T(P) - T(P') &= (n-a+1)(s_a - s_b) + (n-b+1)(s_b - s_a) \\ &= (b-a)(s_a - s_b) > 0 \end{aligned}$$

Se puede obtener el mismo resultado de manera menos formal a partir de la figura 6.7. Al comparar las planificaciones P y P' , se observa que los $a-1$ primeros clientes salen del sistema exactamente al mismo tiempo en ambas planificaciones. Lo mismo sucede para los $n-b$ últimos clientes. Ahora el cliente a sale cuando antes salía el cliente b , mientras que el cliente b sale antes que salía el cliente a , porque $s_b < s_a$. Finalmente, los clientes que son servidos en las posiciones desde $a+1$ hasta $b+1$ también salen antes del sistema, por la misma razón. Por consiguiente, P' es mejor que P en conjunto.

De esta manera, se puede optimizar toda planificación en la que se sirva a un cliente antes que a otro que requiera menos servicio. Las únicas planificaciones que quedan son aquellas que se obtienen poniendo a los clientes por orden no decreciente de tiempo de servicio. Todas estas planificaciones son claramente equivalentes, y por tanto todas son óptimas.

La implementación de este algoritmo es tan sencilla que omitimos los detalles. En esencia, lo único que se necesita es ordenar los clientes por orden de tiempo no decreciente de servicio, lo cual requiere un tiempo que está en $O(n \log n)$. El problema se puede generalizar a un sistema con s servidores, y también el algoritmo: véase el problema 6.20.

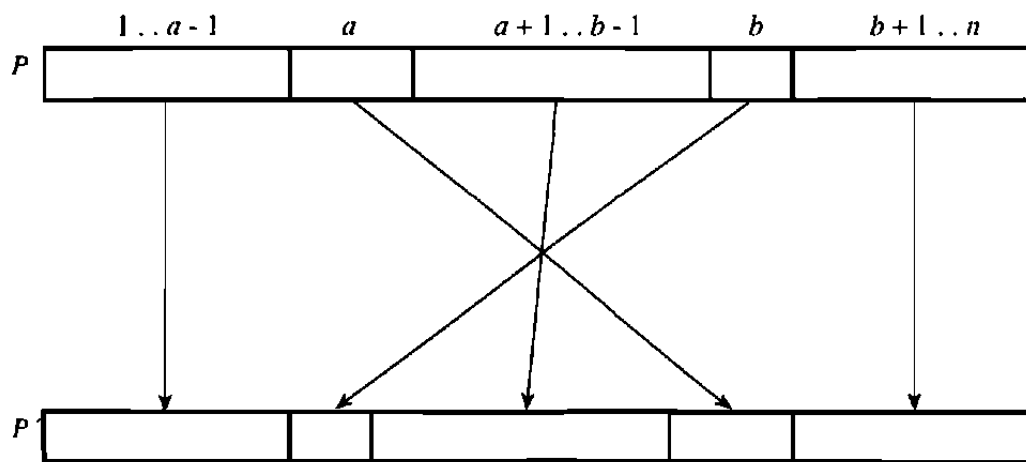


Figura 6.7. Intercambio de dos clientes

6.6.2 Planificación con plazo fijo

Tenemos que ejecutar un conjunto de n tareas, cada una de las cuales requiere un tiempo unitario. En cualquier instante $T = 1, 2, \dots$ podemos ejecutar únicamente una tarea. La tarea i nos produce unos beneficios $g_i > 0$ sólo en el caso de que sea ejecutada en un instante anterior a d_i .

Por ejemplo, con $n = 4$ y los valores siguientes:

i	1	2	3	4
g_i	50	10	15	30
d_i	2	1	2	1

las planificaciones que hay que considerar y los beneficios correspondientes son:

Secuencia	Beneficio	Secuencia	Beneficio
1	50	2,1	60
2	10	2,3	25
3	15	3,1	65
4	30	4,1	80 ← óptimo
1,3	65	4,3	45

Por ejemplo, la secuencia 3,2 no se considera porque la tarea 2 se ejecutaría en el instante $t = 2$, después de su plazo que es $d_2 = 1$. Para maximizar nuestros beneficios en este ejemplo, deberíamos ejecutar la planificación 4,1.

Se dice que un conjunto de tareas es *factible* si existe al menos una sucesión (que también se llama factible) que permite que todas las tareas del conjunto se ejecuten antes de sus respectivos plazos. Un algoritmo voraz evidente consiste en construir la planificación paso a paso, añadiendo en cada paso la tarea que tenga el mayor valor de g_i entre las que aún no se hayan considerado, siempre y cuando el conjunto de tareas seleccionadas siga siendo factible.

En el ejemplo precedente seleccionamos en primer lugar la tarea 1. Después tomamos la tarea 4; el conjunto $\{1, 4\}$ es factible porque se puede ejecutar en el orden 4, 1. A continuación probamos el conjunto $\{1, 3, 4\}$, que resulta no ser factible; por tanto se rechaza la tarea 3. Por último, probamos $\{1, 2, 4\}$, que tampoco es factible, así que se rechaza la tarea 2. Nuestra solución (que en este caso es óptima) es por tanto ejecutar el conjunto de tareas $\{1, 4\}$, que sólo se puede efectuar en el orden 4, 1. Queda por demostrar que este algoritmo siempre encuentra una planificación óptima, y además hay que buscar una forma eficiente de implementarlo.

Sea J un conjunto de k tareas. A primera vista, puede parecer que necesitamos probar las $k!$ permutaciones de estas tareas para ver si J es factible. Afortunadamente, esto no es así.

Lema 6.6.2 Sea J un conjunto de k tareas. Supongamos, sin pérdida de generalidad, que las tareas están numeradas de tal forma que $d_1 \leq d_2 \leq \dots \leq d_k$. Entonces el conjunto J es factible si y sólo si la secuencia 1, 2, ..., k es factible

Demostración. El «si» es evidente. Para el «sólo si», supongamos que la secuencia 1, 2, ..., k no es factible. Entonces al menos una de estas tareas se planifica después de su plazo. Sea r cualquiera de estas tareas, de tal manera que $d_i \leq r - 1$. Dado que las tareas se planifican por orden no decrecientes de plazos, esto significa que al menos r tareas tienen como fecha final $r - 1$ ó anterior. Sea cual fuere la forma en que se planifiquen, la última siempre llegará tarde.

Esto demuestra que basta comprobar una sola secuencia, en orden no decreciente, para saber si un conjunto de tareas J es o no factible.

Teorema 6.6.3 *El algoritmo voraz esbozado anteriormente siempre encuentra una planificación óptima*

Demostración. Supongamos que el algoritmo voraz decide ejecutar un conjunto de tareas I , y supongamos que el conjunto J es óptimo. Sean S_I y S_J secuencias factibles, que posiblemente incluyan huecos, para los dos conjuntos de tareas en cuestión. Reorganizando las tareas de S_I y de S_J , podemos obtener dos secuencias factibles S'_I y S'_J , que también pueden contener huecos, tales que toda tarea común a I y a J se planifique en el mismo instante en ambas secuencias; véase la figura 6.8.

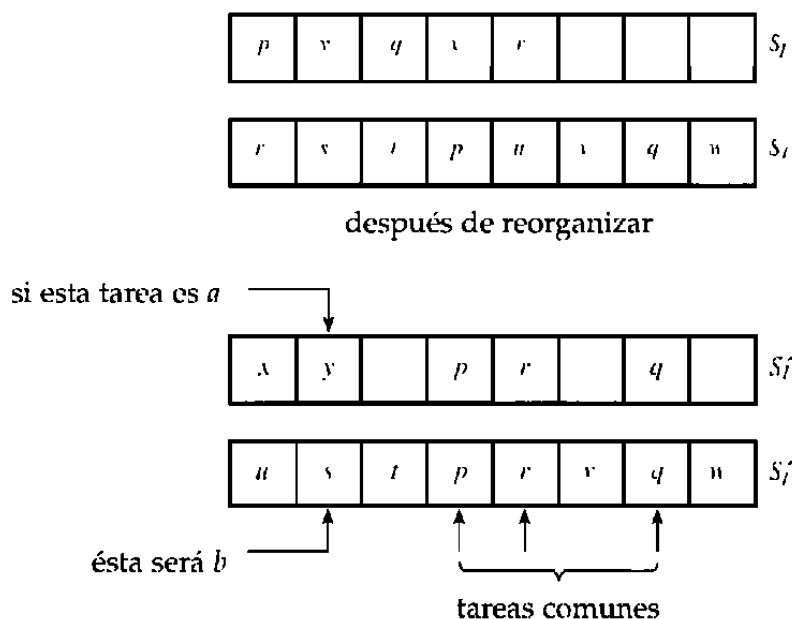


Figura 6.8. Reorganización de tareas para juntar las tareas idénticas

Para ver esto, imaginemos que alguna tarea a aparece en las dos secuencias factibles S_I y S_J , en donde queda planificada en los instantes t_I y t_J , respectivamente. Si $t_I = t_J$, no hay nada que hacer. En caso contrario, supongamos que $t_I < t_J$. Dado que la secuencia S_I es factible, se sigue que el plazo para la tarea a no es anterior a t_I . Se modifica la secuencia S_I de la manera siguiente: si hay un hueco en la secuencia S_I en el instante t_J , se atrasa la tarea a del instante t_I al hueco en el instante t_J ; si hay una tarea b planificada en S_I en el instante t_J , se intercambian las tareas a y b en la secuencia S_I . La secuencia resultante sigue siendo factible, puesto que en cualquier caso a se ejecutará antes de su plazo, y en el segundo caso el traslado de b

a un instante anterior no puede en modo alguno causar daños. Ahora se planifica a en el mismo instante t_i en las dos secuencias modificadas S_i y S_j . Se puede aplicar un argumento similar cuando $t_i > t_j$, salvo que en este caso es S_j quien debe ser modificada.

Una vez que se ha tratado una tarea a de esta manera, está claro que nunca será preciso volver a trasladarla. Por tanto, si las secuencias S_i y S_j tiene m tareas en común, después de un máximo de m modificaciones de S_i o de S_j podemos asegurar que todas las tareas comunes a I y a J estarán planificadas al mismo tiempo en ambas secuencias. Las secuencias resultantes S'_i y S'_j pueden no ser iguales si $I \neq J$. Por tanto, supongamos que existe un instante en el cual la tarea planificada en S_j es distinta de la planificada en S'_i .

- ◇ Si alguna tarea a está planificada en S'_i frente a un hueco de S'_j , entonces a no pertenece a J . El conjunto $J \cup \{a\}$ es factible, porque podríamos poner a en el hueco, y sería más rentable que J . Esto es imposible puesto que J es óptimo por hipótesis.
- ◇ Si alguna tarea b está planificada en S'_j frente a un hueco de S'_i , el conjunto $I \cup \{b\}$ sería factible, así que el algoritmo voraz habría incluido a b en I . Esto también es imposible, porque no lo hizo.
- ◇ La única posibilidad restante es que alguna tarea a esté planificada en S'_i al lado de una tarea distinta b en S'_j . En este caso, a no aparece en J y b no aparece en I . Aparentemente, hay tres posibilidades:
 - Si $g_a > g_b$, se podría sustituir a por b en J y mejorarla. Esto es imposible porque J es óptima.
 - Si $g_a < g_b$, el algoritmo voraz habría seleccionado a b antes de considerar a a , puesto que $(I \setminus \{a\}) \cup \{b\}$ sería factible. Esto también es imposible porque el algoritmo no incluyó a b en I .
 - La única posibilidad restante es que $g_a = g_b$.

Concluimos que para toda posición temporal las secuencias S'_i y S'_j o bien no planifican tareas, o planifican la misma tarea, o planifican dos tareas distintas que producen idéntico beneficio. El beneficio total de I es por tanto igual al beneficio del conjunto óptimo J , así que I también es óptimo.

Para nuestra primera implementación del algoritmo, supondremos sin pérdida de generalidad que las tareas están numeradas de tal manera que $g_1 \geq g_2 \geq \dots \geq g_n$. El algoritmo se puede implementar de forma más eficiente (y más sencilla) si suponemos además que $n > 0$ y $d_i > 0$, para $1 \leq i \leq n$, y que está disponible un espacio adicional al principio de los vectores d (que contiene los plazos) y j (en donde se construye la solución). Estas celdas adicionales se conocen con el nombre de «centinelas». Almacenando un valor adecuado en los centinelas se evitan comprobaciones repetitivas de rangos que consumen mucho tiempo.

```

función secuencia( $d[0..n]$ ):  $k$ , matriz  $j[1..k]$ 
    matriz  $j[0..n]$ 
    {La planificación se construye paso a paso en la matriz  $j$ .
     La variable  $k$  dice cuántas tareas están ya
     en la planificación.}
     $d[0] \leftarrow j[0] \leftarrow 0$  {centinelas}
     $k \leftarrow j[1] \leftarrow 1$  {la tarea 1 siempre se selecciona}
    {bucle voraz}
    para  $i \leftarrow 2$  hasta  $n$  hacer {orden decreciente de  $g$ }
         $r \leftarrow k$ 
        mientras  $d[j[r]] > \text{máx}(d[i], r)$  hacer  $r \leftarrow r - 1$ 
        si  $d[i] > r$  entonces
            para  $m \leftarrow k$  paso  $-1$  hasta  $r + 1$  hacer  $j[m + 1] \leftarrow j[m]$ 
             $j[r + 1] \leftarrow i$ 
             $k \leftarrow k + 1$ 
    devolver  $k$ ,  $j[1..k]$ 

```

Las k tareas de la matriz j están por orden creciente de plazo. Cuando se está considerando la tarea i , el algoritmo comprueba si se puede insertar en j en el lugar oportuno sin llevar alguna tarea que ya esté en j más allá de su plazo. De ser así, i se acepta; en caso contrario, i se rechaza. Los valores exactos de los g_i son innecesarios siempre y cuando las tareas estén numeradas correctamente por orden decreciente de beneficios. La figura 6.9 da g_i y d_i para un ejemplo de seis tareas, y la figura 6.10 ilustra la forma en que funciona el algoritmo en este ejemplo. (La figura 6.10 lo llama algoritmo «lento» porque describiremos uno mejor dentro de poco.)

i	1	2	3	4	5	6
g_i	20	15	10	7	5	3
d_i	3	1	1	3	1	3

Figura 6.9. Un ejemplo con seis tareas

El análisis del algoritmo es sencillo. La ordenación de las tareas por orden decreciente de beneficio requiere un tiempo que está en $\Theta(n \log n)$. El caso peor para el algoritmo es cuando este procedimiento también clasifica las tareas por orden decreciente de plazos, y cuando todas ellas tienen cabida en la planificación. En este caso, cuando se está considerando la tarea i el algoritmo examina las $k = i - 1$ tareas que ya están planificadas, para encontrar un lugar para el recién llegado, y después las desplaza todas un lugar. En términos del programa anterior, hay $\sum_{i=1}^{n-1} k$ pasadas por el bucle **mientras** y $\sum_{m=1}^{n-1} m$ pasadas por el bucle **para** interno. El algoritmo requiere, por tanto, un tiempo que está en $\Omega(n^2)$.

Se obtiene un algoritmo más eficiente cuando se utiliza una técnica distinta para verificar si es factible un conjunto de tareas dado. La nueva técnica depende del lema siguiente.

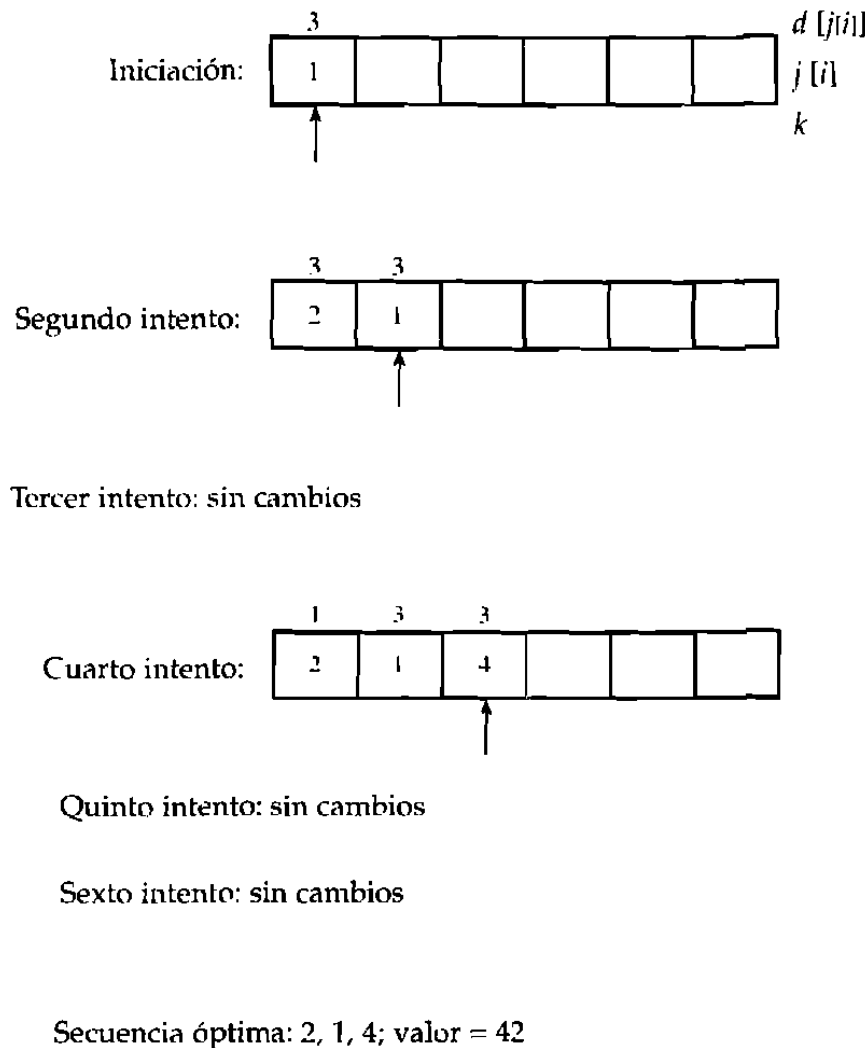


Figura 6.10. Ilustración del algoritmo lento

Lema 6.6.4 Un conjunto J de n tareas es factible si y sólo si se puede construir una secuencia factible que incluya a todas las tareas de J en la forma siguiente. Se empieza con una planificación vacía, de longitud n . Entonces, para cada tarea $i \in J$ sucesivamente, se planifica i en el instante t , en donde t es el mayor entero tal que $1 \leq t \leq \min(n, d_i)$, y la tarea que se ejecutará en el instante t no está decidida todavía.

En otras palabras, se empieza por una planificación vacía, se considera cada tarea sucesivamente, y se añade a la planificación que se está construyendo en el momento más tardío posible, pero no antes de su fecha final. Si no se puede planificar una tarea antes de su plazo, entonces el conjunto J no es factible.

Demostración. El «si» es evidente. Para el «sólo si», obsérvese primero que si existe una secuencia factible, entonces existe una secuencia factible de longitud n . Puesto que sólo hay n tareas por planificar, toda secuencia más larga tendrá que contener huecos, y siempre se puede trasladar una tarea a un hueco anterior sin afectar a la factibilidad de la secuencia.

Cuando se intenta añadir una nueva tarea, la secuencia que se está construyendo contiene siempre al menos un hueco. Supongamos que no se puede añadir una tarea cuyo plazo sea d . Esto puede suceder solamente si todas las posiciones desde $t = 1$ hasta $t = r$ están ya reservadas, en donde $r = \min(n, d)$. Sea $s > r$ el menor entero tal que la posición $t = s$ está vacía. La planificación que ya se ha construido incluye por tanto $s - 1$ tareas, ninguna tarea con plazo exactamente s y quizá otras más con plazos posteriores a s . La tarea que estamos intentando añadir también tiene un plazo anterior a s . Por tanto J contiene al menos s tareas cuyas plazos son $s - 1$ o anteriores. Sea cual fuere la forma en que se planifiquen, la última llegará tarde con certeza.

El lema sugiere que deberíamos considerar un algoritmo que intente llenar una por una las posiciones de una secuencia de longitud p , donde $p = \min(n, \max_{1 \leq i \leq n} d_i)$. Para cualquier posición t , se define $n_t = \max\{k \leq t \mid \text{la posición } k \text{ no está ocupada}\}$. También se definen ciertos conjuntos de posiciones en la forma siguiente: dos posiciones i y j están en el mismo conjunto si $n_i = n_j$; véase la figura 6.11. Para un conjunto dado K de posiciones, sea $F(K)$ el menor elemento de K . Finalmente, se define una posición ficticia cero, que siempre está libre.

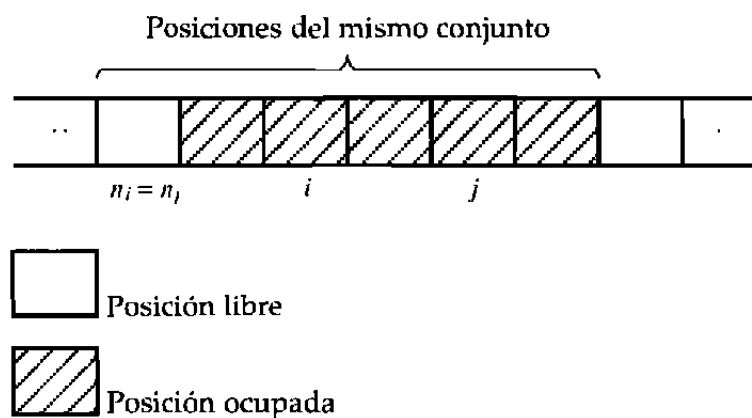


Figura 6.11 Conjuntos de posiciones

A medida que se asignan nuevas tareas a posiciones vacantes, los conjuntos se fusionan para formar conjuntos más grandes: las estructuras de partición están diseñadas precisamente con este propósito. Obtenemos un algoritmo cuyos pasos esenciales son los siguientes:

- (i) Iniciación: Toda posición $0, 1, 2, \dots, p$ está en un conjunto diferente y $F(\{i\}) = i$, $0 \leq i \leq p$.
- (ii) Adición de una tarea con plazo d : se busca el conjunto que contenga a d ; sea K este conjunto. Si $F(K) = 0$ se rechaza la tarea; en caso contrario:
- Se asigna la nueva tarea a la posición $F(K)$.
 - Se busca el conjunto que contenga $F(K) - 1$. Llamemos L a este conjunto (no puede ser igual a K).
 - Se fusionan K y L . El valor de F para este nuevo conjunto es el valor viejo de $F(L)$.

La figura 6.12 ilustra el funcionamiento de este algoritmo para el ejemplo dado en la figura 6.9.

Lo que sigue es un enunciado más preciso del algoritmo rápido. Para simplificar la descripción, suponemos que la etiqueta del conjunto producido por una operación de *fusionar* es necesariamente la etiqueta de uno de los conjuntos que hayan sido fusionados. La planificación producida en primer lugar puede contener huecos; el algoritmo acaba por trasladar tareas hacia adelante para llenarlos:

```

función secuencia2( $d[1..n]$ ):  $k$ ; matriz  $[1..k]$ 
    matriz  $j$ ,  $F[0..n]$ 
    {Iniciación}
     $p = \min. (n, \max\{d[i] \mid 1 \leq i \leq n\})$ 
    para  $i \leftarrow 0$  hasta  $p$  hacer    $j[i] \leftarrow 0$ 
                                      $F[i] \leftarrow i$ 
                                     iniciar el conjunto  $\{i\}$ 

    {Bucle voraz}
    para  $i \leftarrow 1$  hasta  $n$  hacer {orden decreciente de  $g$ }
         $k \leftarrow \text{buscar}(\min(p, d[i]))$ 
         $m \leftarrow F[k]$ 
        si  $m \neq 0$  entonces
             $j[m] \leftarrow i$ 
             $l \leftarrow \text{buscar}(m - 1)$ 
             $F[k] \leftarrow F[l]$ 
            fusionar( $k, l$ ) {el conjunto resultante tiene la etiqueta  $k$  o  $l$ }

    {sólo queda comprimir la solución}
     $k \leftarrow 0$ 
    para  $i \leftarrow 1$  hasta  $p$  hacer
        si  $j[i] > 0$  entonces    $k \leftarrow k + 1$ 
                                 $j[k] \leftarrow j[i]$ 

    devolver  $k, j[1..k]$ 

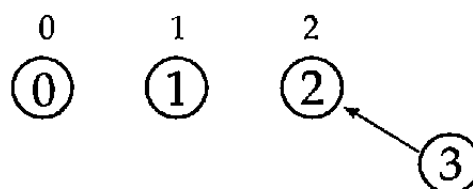
```

Si se nos da el problema con las tareas ya ordenadas por beneficios decrecientes, de tal modo que sea posible obtener una secuencia óptima sin más que llamar al algoritmo anterior, entonces la mayor parte del conjunto se invertirá en manipular conjuntos disjuntos. Dado que hay que ejecutar como máximo $2n$ operaciones *buscar* y n operaciones *fusionar*, el tiempo requerido está en $O(n\alpha(2n, n))$, donde α es la función de crecimiento lento de la Sección 5.9. Esto es esencialmente lineal. Si, por otra parte, las tareas se nos dan por un orden arbitrario, entonces tendremos que empezar por ordenarlas, y la obtención de la secuencia inicial requiere un tiempo que está en $\Theta(n \log n)$.

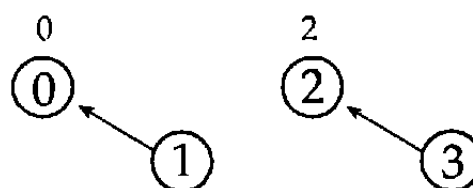
Iniciación: $p = \min(6, \max(d_i)) = 3$



Intento 1: $d_1 = 3$, se asigna la tarea 1 a la posición 3

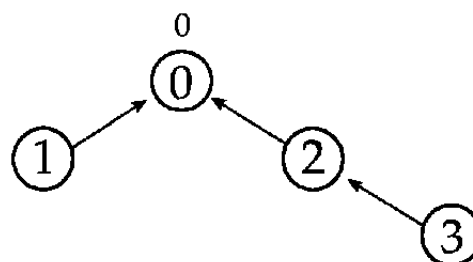


Intento 2: $d_2 = 1$, se asigna la tarea 2 a la posición 1



Intento 3: $d_3 = 1$ no hay posiciones libres disponibles porque el valor de F es 0

Intento 4: $d_4 = 3$, se asigna la tarea 4 a la posición 2



Intento 5: $d_5 = 1$, no hay posiciones libres disponibles

Intento 6: $d_6 = 3$, no hay posiciones libres disponibles

Secuencia óptima: 2, 4, 1; valor = 42

Figura 6.12. Ilustración del algoritmo rápido

6.7 PROBLEMAS

Problema 6.1. ¿Es la ordenación por selección (véase la Sección 2.4) un algoritmo voraz? De ser así, ¿cuáles son las distintas funciones necesarias (la función para comprobar la factibilidad, la función de selección y las demás)?

Problemas 6.2. El sistema monetario inglés, antes de la normalización decimal, constaba de medias coronas (30 peniques), florines (24 peniques), chelines (12 peniques), monedas de seis peniques, tres peniques y peniques (por no mencionar los medios peniques y los *farthings*³, que valían respectivamente medio y un cuarto de penique respectivamente). Demostrar que con estas monedas el algoritmo voraz de la Sección 6.1 no produce necesariamente una solución óptima, aun cuando se disponga de un suministro inagotable de monedas de cada valor.

Problema 6.3. El sistema monetario portugués consta de monedas de 1, 2'50, 5, 10, 20, 25 y 50 escudos. Sin embargo, los precios son siempre un número entero de escudos. Demostrar lo siguiente, o dar un contraejemplo: cuando se dispone de un suministro inagotable de monedas de cada valor, el algoritmo voraz de la Sección 6.1 siempre encuentra una solución óptima.

Problema 6.4. Supongamos que el sistema monetario consta de los valores dados en la Sección 6.1, pero que nos quedamos sin duros. Mostrar que al utilizar el algoritmo voraz con los valores restantes no se llega necesariamente a una solución óptima.

Problema 6.5. Demostrar lo siguiente, o dar un contraejemplo: supuesto que toda moneda de la serie valga al menos el doble que la moneda de valor inmediatamente inferior, y

que la serie contiene una moneda de valor unitario, y se dispone de un suministro inagotable de monedas de cada valor, entonces el algoritmo voraz de la Sección 6.1 siempre encuentra una solución óptima.

Problema 6.6. Supongamos que los valores de monedas disponibles son de la forma $1, p, p^2, \dots, p^n$ en donde $p > 1$ y $n \geq 0$ son enteros. Demostrar lo siguiente, o dar un contraejemplo: con esta serie de valores y un suministro inagotable de monedas de cada valor, el algoritmo voraz de la Sección 6.1 siempre encuentra una solución óptima.

Problema 6.7. Demostrar que un grafo con n nodos y más de $n - 1$ aristas debe contener al menos un ciclo.

Problema 6.8. Supongamos que el coste de tender una línea telefónica desde el punto a hasta el punto b es proporcional a la distancia euclidiana entre a y b . Es preciso unir entre sí un cierto número de ciudades con el mínimo coste. Buscar un ejemplo en el que cueste menos tender los cables a través de una centralita situada entre ciudades que utilizar solamente enlaces directos de conexión entre ciudades.

Problema 6.9. ¿Qué se puede decir acerca del tiempo requerido por el algoritmo de Kruskal si, en lugar de proporcionar una lista de aristas, el usuario proporciona una matriz de distancias, dejando para el algoritmo la tarea consistente en determinar las aristas que existen?

Problema 6.10. Supongamos que se implementan los algoritmos de Kruskal y el de Prim según se muestra en las Secciones 6.3.1 y 6.3.2 respectivamente. ¿Qué sucede (a) en el caso del algoritmo de Kruskal y (b) en el caso del algoritmo de Prim si por error se ejecuta el algoritmo sobre un grafo que no es conexo?

Problema 6.11. Un grafo puede tener varios árboles de recubrimiento mínimo diferentes. ¿Es esto cierto para el grafo de la figura 6.2? De ser así, ¿dónde se refleja esta posibilidad en los algoritmos explicados en las Secciones 6.3.1 y 6.3.2?

Problema 6.12. El problema consistente en hallar un subconjunto T de las aristas de un grafo conexo G tal que todos los nodos queden conectados utilizando solamente las aristas de T , y cuando la suma de las longitudes de las aristas de T sea la más pequeña posible, sigue teniendo sentido aun cuando G tenga aristas con longitudes negativas. Sin embargo, la solución quizá ya no sea un árbol. Adaptar bien el algoritmo de Kruskal o bien el de Prim para que funcionen en un grafo que pueda incluir aristas de longitud negativa.

Problema 6.13. Demostrar que el algoritmo de Prim, al igual que el algoritmo de Kruskal, se puede implementar utilizando montículos. Demostrar que en tal caso requiere un tiempo que está en $\Theta(a \log n)$.

Problema 6.14. En el algoritmo de Dijkstra, cuando se añade un nuevo nodo v a S , sea w un nodo que no está en S . ¿Es posible que el nuevo camino especial más corto desde el origen hasta w tenga que pasar primero por v y después por algún otro nodo de S ?

Problema 6.15. Demostrar, dando un ejemplo explícito, que si las longitudes de las aristas pueden ser negativas, entonces el algoritmo de Dijkstra no siempre funciona correctamente. ¿Siguiendo teniendo sentido hablar del camino más corto si se admiten distancias negativas?

Problema 6.16. En el análisis de la implementación del algoritmo de Dijkstra que em-

plea un montículo, se vio que se pueden hacer flotar hasta n nodos, mientras que se eliminan menos de n raíces. La eliminación de la raíz tiene como consecuencia la inmersión del nodo que ocupa su lugar. En general, hacer flotar resulta ligeramente más rápido que sumergir, puesto que en cada nivel se compara el valor del nodo con el valor de su padre, en lugar de hacer comparaciones con los dos hijos. Empleando un montículo k -ario (véase la Sección 5.7 y el Problema 5.23), se puede conseguir que la flotación funcione todavía más deprisa, a costa de hacer más lenta la inmersión. Sea $k = \max(2, \lfloor a / n \rfloor)$. Demostrar la forma de utilizar un montículo k -ario para calcular las rutas más cortas desde un origen hasta todos los demás nodos de un grafo en un tiempo $O(a \log_k n)$. Observe que esto da $O(n^2)$ si $a \approx n^2$ y $O(a \log n)$ si $a \approx n$. Por tanto, nos da lo mejor de ambos mundos.

Problema 6.17. Los montículos de Fibonacci, que se mencionan en la Sección 5.8, tienen las propiedades siguientes. Un montículo que contiene n elementos se puede construir en un tiempo $\Theta(n)$; la búsqueda del mayor elemento, la inserción de un nuevo elemento, el incremento del valor de un elemento y la restauración de la propiedad del montículo, y la fusión de dos montículos requieren todos ellos un tiempo amortizado que está en $\Theta(1)$, y el borrado de cualquier elemento, incluyendo en particular el mayor de ellos, a partir de un montículo que contenga n elementos, requiere un tiempo amortizado que está en $O(\log n)$. Los montículos invertidos de Fibonacci son similares, salvo que las operaciones correspondientes consisten en disminuir el valor de un elemento y buscar o eliminar el elemento más pequeño. Mostrar la forma en que se puede utilizar un montículo invertido de Fibonacci para implementar el algoritmo de Dijkstra en un tiempo que esté en $O(a + n \log n)$.

Problema 6.18. En la Sección 6.5 se suponía que estaban disponibles n objetos, numerados de 1 a n . Supongamos en cambio que tenemos n tipos de objetos disponibles, con un suministro adecuado de objetos de cada tipo. Formalmente, esto sólo cambia la restricción anterior $0 \leq x_i \leq 1$ por la restricción más permisiva $x_i \geq 0$. ¿Sigue funcionando el algoritmo voraz de la Sección 6.5? ¿Sigue siendo necesario?

Problema 6.19. Demostrar lo siguiente o dar un contraejemplo: para el problema de planificación con plazos de la Sección 6.6.1, la planificación de clientes por orden descendente de tiempos de servicio da lugar a la peor planificación posible.

Problema 6.20. Como en la Sección 6.6.1, tenemos n clientes. El cliente i , para $1 \leq i \leq n$, requiere un tiempo de servicio conocido t_i . Sin pérdida de generalidad, supongamos que los clientes están numerados de tal manera que $t_1 \leq t_2 \leq \dots \leq t_n$. Si hay s servidores idénticos, demostrar que para minimizar el tiempo total (y por tanto el tiempo medio) pasado en el sistema por los clientes, el servidor j , $1 \leq j \leq s$, debe dar servicio a los clientes $j, j + s, j + 2s, \dots$ por ese orden.

Problema 6.21. Sean n programas $P_1, P_2, \dots, P_n$ que hay que almacenar en un disco. El programa P_i requiere s_i kilobytes de espacio, y la capacidad del disco es D kilobytes, donde $D < \sum_{i=1}^n s_i$.

(a) Se desea maximizar el número de programas almacenados en el disco. Demostrar lo siguiente, o dar un contraejemplo: podemos utilizar un algoritmo voraz que seleccione los programas por orden no decreciente de s_i .

(b) Se desea utilizar la mayor capacidad posible del disco. Demostrar lo siguiente o

dar un contraejemplo: se puede utilizar un algoritmo voraz que seleccione los programas por orden no creciente de s_i .

Problema 6.22. Sean n programas $P_1, P_2, \dots, P_n$ que hay que almacenar en una cinta. El programa P_i requiere s_i kilobytes de espacio; la tapa es suficientemente larga para almacenar todos los programas. Sabemos con qué frecuencia se utiliza cada programa: una fracción π_i de las solicitudes afecta al programa i (y por tanto $\sum_{i=1}^n \pi_i = 1$). La información en la cinta es almacenada (grabada) con densidad constante, y la velocidad de la cinta también es constante. Una vez que se carga el programa, la cinta se rebobina hasta el principio. Si los programas se almacenan por orden $i_1, i_2, \dots, i_n$ el tiempo medio requerido para cargar un programa es, por tanto:

$$\bar{T} = c \sum_{i=1}^n \left[\pi_{i_1} \sum_{k=1}^{i_1} s_{i_k} \right]$$

donde la constante c depende de la densidad de grabación y de la velocidad de la cinta. Deseamos minimizar $\bar{T}$ empleando un algoritmo voraz. Demostrar todo lo siguiente, o dar un contraejemplo: podemos seleccionar los programas (a) por orden de s_i no decreciente; (b) por orden de π_i no creciente; (c) por orden de π_i/s_i no creciente.

Problema 6.23. Supongamos que las dos planificaciones S_I y S_J presentadas en la demostración de optimalidad de la Sección 6.6.2 se dan en la forma de matrices $SI[1..r]$ y $SJ[1..r]$, donde $r = \max_{1 \leq i \leq n} d_i$. Un elemento de la matriz contiene i si la tarea i debe ejecutarse en el momento correspondiente, y cero representa un hueco en la planificación. Escribir un algoritmo que produzca las planificaciones S'_I y S'_J en las matrices SI y SJ respectivamente.